G. L. BAJAJ GROUP OF INSTITUTIONS, MATHURA
Department of Computer Science & Engineering • B.Tech 5th Sem, 2026–27 Session

CAPSTONE RESEARCH
MTC PROJECT

Research Study Notes & Review Portfolio

Student Name:	**Ajay Sisodia** (Roll No: `2405111530012`)
Research Track:	**Track A: Foundational / Accessible**
Assigned Domain:	TinyML Systems Architecture, Microcontroller Inference Engines, Benchmarking & Edge Deployment (Papers 1–5)
Project:	Micro-Train-Compiler (MTC): On-Device NN Training on Microcontrollers

Complete Handwritten-Style Study Notes for Student 1

Student: Ajay Sisodia (Roll No: 2405111530012)

Subject: TinyML & Embedded Systems Architecture

Project: Micro-Train-Compiler (MTC)

Focus Domain: TinyML Systems Architecture, Microcontroller Inference Engines, Benchmarking & Edge Deployment

---

Note for Ajay:
These notes are formatted for **100% clean rendering in VS Code, Markdown preview extensions, and GitHub**. Display equations are formatted in standard KaTeX / MathJax blocks with zero indentation, and inline values use universal symbols (≤, ≥, ×, √) so they look great both in digital preview and when copied into your handwritten notebook.

TABLE OF CONTENTS

1. Paper 1: TensorFlow Lite Micro (TFLM) — Embedded ML on TinyML Systems (MLSys 2021)
2. Paper 2: Tiny Machine Learning: Progress and Futures (IEEE 2024 / Survey)
3. Paper 3: CMSIS-NN — Efficient Neural Network Kernels for Arm Cortex-M CPUs (Arm Research)
4. Paper 4: Benchmarking TinyML Systems: Challenges and Directions (MLPerf Tiny)
5. Paper 5: Edge Intelligence: Paving the Last Mile of AI with Edge Computing (IEEE Survey)
6. Quick Reference Summary Table for Ajay

Paper 1: TensorFlow Lite Micro (TFLM) — Embedded ML on TinyML Systems (MLSys 2021)

- **Authors:** Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Pete Warden, et al. (Google Brain & Harvard)
- **Conference:** MLSys 2021
- **Direct Link:** https://arxiv.org/abs/2010.08678
- **Direct PDF:** https://arxiv.org/pdf/2010.08678.pdf

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** Standard mobile deep learning frameworks (like mobile TensorFlow Lite or ONNX Runtime) depend on an Operating System (Linux, Android), hardware floating-point units (FPU), and dynamic heap memory (`malloc()` and `free()`). On microcontrollers (like ESP32 or Cortex-M4) with only a few hundred kilobytes of RAM and no OS, standard runtimes cause memory fragmentation, unpredictable latency, and crashes.

- **The Breakthrough Idea:** TensorFlow Lite Micro (TFLM) is Google's interpreter-based runtime designed strictly for bare-metal microcontrollers with:
  1. **Zero Dynamic Allocation:** Eliminates all calls to `malloc()` at runtime. All tensor memory is allocated upfront out of a single pre-allocated contiguous memory buffer called the **Tensor Arena**.
  2. **No OS Dependency:** Pure C++ core that compiles directly on bare-metal hardware without Linux or POSIX threads.
  3. **Modular Kernel Resolver:** Only links the specific neural network operator kernels needed by the model (`MicroMutableOpResolver`), keeping the compiled binary footprint under 20 KB.

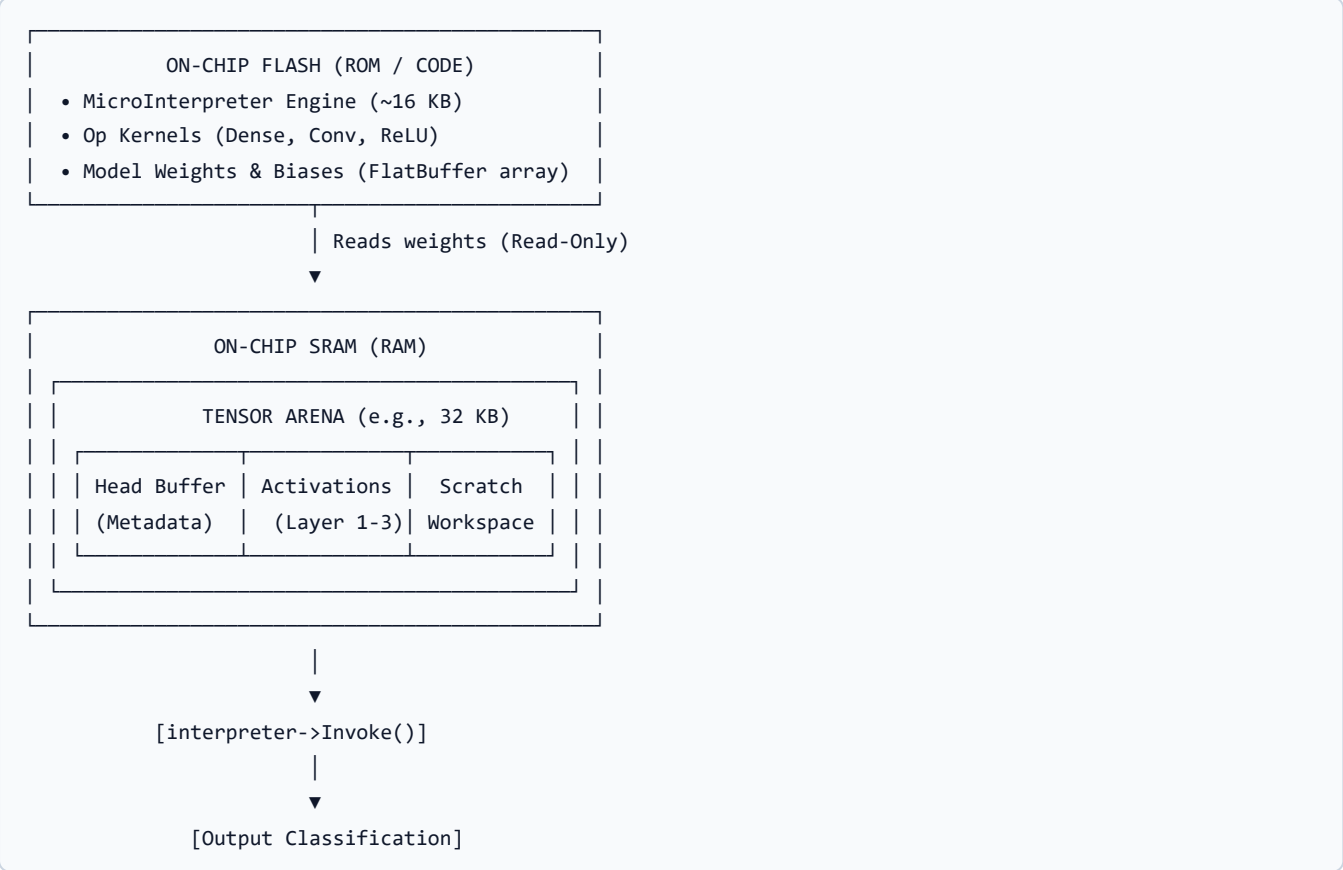
---

2. Step-by-Step Workflow

1. **Model Export:** A neural network trained in TensorFlow is quantized (INT8) and exported as a `.tflite` FlatBuffer binary.
2. **C Byte Array Conversion:** The `.tflite` file is converted into a C byte array (`model_data.cc`) and burned directly into on-chip Flash memory.
3. **Tensor Arena Allocation:** The developer allocates a single static byte array in SRAM: `uint8_t tensor_arena[ARENA_SIZE];`
4. **Op Resolver Registration:** The developer registers only the required kernels (e.g., `AddFullyConnected()`, `AddRelu()`).
5. **Initialization & Planning:** `MicroInterpreter` parses the FlatBuffer, plans tensor offsets inside the arena, and verifies bounds.
6. **Inference Execution:** For every sensor reading, input data is copied to the input tensor and `interpreter->Invoke()` executes the kernels sequentially.

---

3. Diagram to Draw in Your Notebook



---

4. Key Formulas & Methodology

1. Contiguous Static Arena Memory Footprint:

$$M_{\text{SRAM}} = M_{\text{metadata}} + M_{\text{activations}} + M_{\text{scratch}}$$

- *Explanation:* Unlike desktop systems where each layer requests memory from the OS, TFLM enforces a rigid upper bound M_{SRAM} that cannot grow during execution, eliminating heap fragmentation.

2. Affine INT8 Dequantization Formula:

$$r = S \cdot (q - Z)$$

- *Explanation:* Microcontroller integer activations $q \in [-128, 127]$ are mapped to real floating values r using scalar scale S and zero-point integer Z.

---

5. Viva / Exam Questions for Ajay

- **Q: Why can't standard TensorFlow Lite run directly on an ESP32 without TFLM?**
  Answer: Standard TFLite relies on POSIX operating system features, multi-threading, dynamic memory allocation (`malloc`), and standard C++ libraries that do not exist on bare-metal microcontrollers.
- **Q: What is the Tensor Arena in TFLM?**
  Answer: It is a single, pre-allocated contiguous array in SRAM where TFLM stores all model activations, intermediate states, and internal metadata, ensuring zero dynamic memory fragmentation.
- **Q: What is the major limitation of TFLM that our MTC project solves?**
  Answer: TFLM is strictly an **inference-only** engine; it has zero capability for on-device backpropagation or weight updates. MTC implements full on-device training.

Paper 2: Tiny Machine Learning: Progress and Futures (IEEE 2024 / Survey)

- **Authors:** Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chung Wang, Song Han (MIT HAN Lab)
- **Venue:** IEEE Circuits and Systems Magazine / arXiv 2024
- **Direct Link:** https://arxiv.org/abs/2403.19076
- **Direct PDF:** https://arxiv.org/pdf/2403.19076.pdf
- **Companion Reference:** Partha Pratim Ray, *A Review on TinyML: State-of-the-Art and Prospects*, Journal of King Saud University, 2022 (DOI: 10.1016/j.jksuci.2021.11.019)

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** Cloud-based machine learning requires streaming raw sensor data (audio, motion, camera feeds) over Wi-Fi or cellular networks to remote servers. This introduces three massive bottlenecks: high latency (100–500 ms), high battery drain (RF transmission consumes 10× to 100× more energy than compute), and severe privacy violations (sensitive home/medical audio leaving the premises).
- **The Breakthrough Idea:** TinyML enables intelligence directly at the analog/digital sensor interface on battery-operated microcontrollers consuming less than 1 milliwatt of power. The paper reviews the technological revolution from simple keyword spotting to complex visual perception and highlights the critical frontier: transitioning from static inference to **on-device continual learning**.

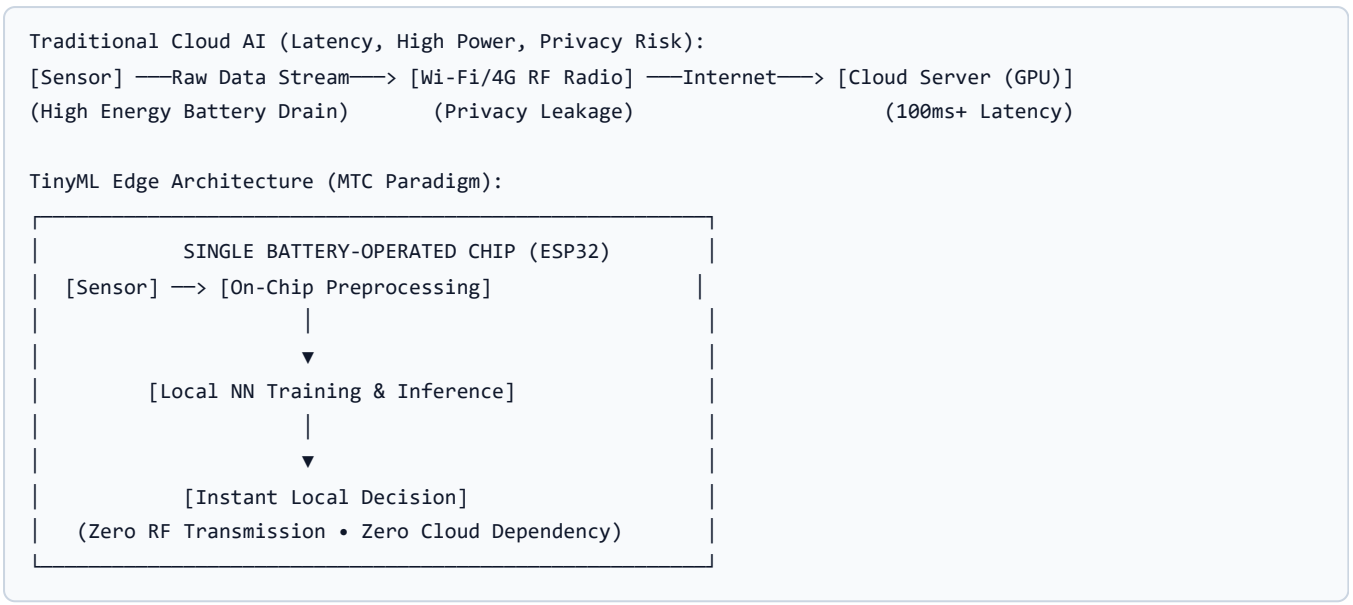
---

2. Step-by-Step Workflow

1. **Sensor Ingestion:** Analog micro-electromechanical sensors (MEMS accelerometers, microphones) sample environmental data.
2. **Local Preprocessing:** On-chip DSP filters raw signals into spectrograms or normalized feature vectors.
3. **On-Chip Inference:** Compact quantized neural networks execute on the microcontroller core within 10–50 ms.
4. **Local Anomaly Detection:** The device evaluates output probabilities locally without radio transmission.
5. **Selective Adaptation / Actuation:** If environmental drift or user habits change, the device adapts its weights on-chip or activates an actuator, keeping the radio sleeping 99% of the time.

---

3. Diagram to Draw in Your Notebook



---

4. Key Formulas & Methodology

1. Edge Energy Consumption Advantage:

$$E_{\text{total}} = E_{\text{compute}} + E_{\text{trans}} \quad \text{where } E_{\text{trans}} \gg E_{\text{compute}}$$

- *Explanation:* Transmitting 1 bit over Wi-Fi/Bluetooth requires roughly 10 to 100 nanojoules, whereas computing a 32-bit MAC operation on-chip requires only 10 to 50 picojoules. Keeping computation on the MCU saves orders of magnitude of battery energy.

2. Microcontroller Hard Resource Invariants:

$$\text{Flash} \leq 4\ \text{MB}, \quad \text{SRAM} \leq 520\ \text{KB}, \quad P_{\text{active}} \leq 100\ \text{mW}$$

- *Explanation:* Unlike servers with terabytes of storage, microcontroller architectures must respect rigid physical memory boundaries.

---

5. Viva / Exam Questions for Ajay

- **Q: What is the primary difference between Edge AI and TinyML?**
  Answer: Edge AI typically refers to devices with Linux OS and gigabytes of RAM (like Raspberry Pi or NVIDIA Jetson) consuming watts of power. TinyML refers strictly to microcontrollers (MCU) with kilobytes of RAM consuming milliwatts of power.
- **Q: Why does local edge processing save battery compared to sending data to the cloud?**
  Answer: Wireless radio transmission (Wi-Fi, LTE, BLE) consumes significantly more energy per bit than executing arithmetic operations directly on the local microcontroller CPU.

Paper 3: CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs (Arm Research)

- **Authors:** Liangzhen Lai, Naveen Suda, Vikas Chandra (Arm Research)
- **Venue:** arXiv preprint, 2018
- **Direct Link:** https://arxiv.org/abs/1801.06601
- **Direct PDF:** https://arxiv.org/pdf/1801.06601.pdf

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** General-purpose C matrix multiplication functions (`for(i) for(j) for(k) c[i][j] += a[i][k] * b[k][j]`) are extremely slow on microcontrollers. Microcontrollers often lack floating-point units (FPU) or out-of-order execution, resulting in excessive clock cycles per multiply-accumulate (MAC).
- **The Breakthrough Idea:** Arm developed **CMSIS-NN**, a collection of handcrafted, assembly-optimized C neural network kernels that achieve up to **4.6× speedup and 4.9× energy efficiency improvement** on embedded processors. It accomplishes this through:
  1. **Fixed-Point Q7 and Q15 Representation:** Converting floating-point values into 8-bit and 16-bit integers.
  2. **SIMD (Single Instruction Multiple Data):** Packing two 16-bit multiplications into a single 32-bit instruction (`__SMLAD`).
  3. **Loop Unrolling:** Unrolling matrix loops by 2× and 4× to keep the processor pipeline full.

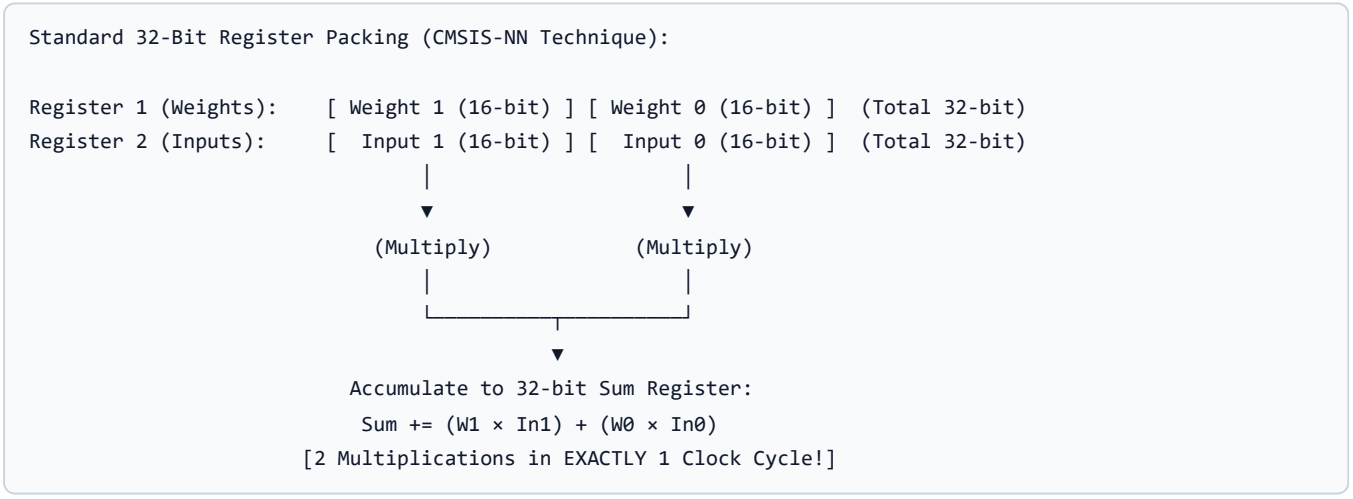
---

2. Step-by-Step Workflow

1. **Model Weights Quantization:** Neural network weights are quantized to symmetric 8-bit (Q7) or 16-bit (Q15) integers.
2. **Matrix Reordering (Im2Col / Transposition):** Feature maps and weights are formatted in memory so consecutive elements can be fetched in 32-bit words.
3. **Dual MAC Execution:** The CPU loads two 16-bit inputs and two 16-bit weights simultaneously using a single 32-bit load instruction.
4. **32-Bit Accumulation:** Multiplications are accumulated into a 32-bit integer accumulator to prevent bit overflow.
5. **Scale & Saturation:** The 32-bit accumulator is shifted down and clamped (saturated) back to the 8-bit or 16-bit target output range.

---

3. Diagram to Draw in Your Notebook



---

4. Key Formulas & Methodology

1. Dual Multiply-Accumulate Arithmetic:

$$ \text{Acc}{32} = |text{Acc}{32} + (W_0 \times x_0) + (W_1 \times x_1) $$

- *Explanation:* On Cortex-M and Xtensa cores, instructions like `SMLAD` execute two 16-bit signed multiplications and accumulate into a 32-bit register in a single machine cycle.

2. Fixed-Point Arithmetic and Saturated Shift:

$$y_{\text{fixed}} = \text{clip}\left((\text{Acc}_{32} + \text{round_offset}) \gg \text{shift_bits}, -32768, 32767 \right)$$

- *Explanation:* To return from a 32-bit accumulator to a 16-bit Q15 representation, the accumulator is right-shifted and clamped to prevent numerical wraparound.

---

5. Viva / Exam Questions for Ajay

- **Q: What is the benefit of fixed-point arithmetic over 32-bit floating point on microcontrollers?**
  Answer: Fixed-point integer arithmetic runs on cheap integer ALUs without requiring power-hungry FPUs, cuts memory usage in half (16-bit vs 32-bit), and executes in fewer CPU clock cycles.
- **Q: How does MTC apply the principles of CMSIS-NN?**
  Answer: MTC emits standalone ANSI C99 code with unrolled inner loops, static 1D array indexing, and fixed-point Q15 arithmetic, ensuring maximum execution speed on ESP32 without relying on heavy external runtime libraries.

Paper 4: Benchmarking TinyML Systems: Challenges and Directions (MLPerf Tiny)

- **Authors:** Colby Banbury, Vijay Janapa Reddi, Max Lam, William Fu, et al. (Harvard & MLPerf Consortium)
- **Venue:** MLPerf Tiny / arXiv 2020
- **Direct Link:** https://arxiv.org/abs/2003.04821
- **Direct PDF:** https://arxiv.org/pdf/2003.04821.pdf

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** In server-side deep learning, models are evaluated purely on accuracy (Top-1%) or throughput (images/sec). On microcontrollers, a model with 99% accuracy is completely useless if it requires 600 KB of SRAM when the chip only has 520 KB, or if it drains the battery in 2 hours. Prior to this paper, there was no standardized, trustworthy way to benchmark and compare TinyML systems.
- **The Breakthrough Idea:** The MLPerf Tiny Consortium established the world's **first open, rigorous benchmarking suite specifically for microcontrollers**. It defines:
  1. **Standardized Tasks:** Keyword Spotting (KWS), Visual Wake Words (VWW), Image Classification (IC), and Anomaly Detection (AD).
  2. **Multi-Dimensional Metrics:** Measuring latency (ms), energy per inference (microjoules), peak SRAM usage (KB), and Flash binary footprint (KB).
  3. **Strict Reproducibility Rules:** Mandating hardware-in-the-loop timing measurements verified via oscilloscope/UART.

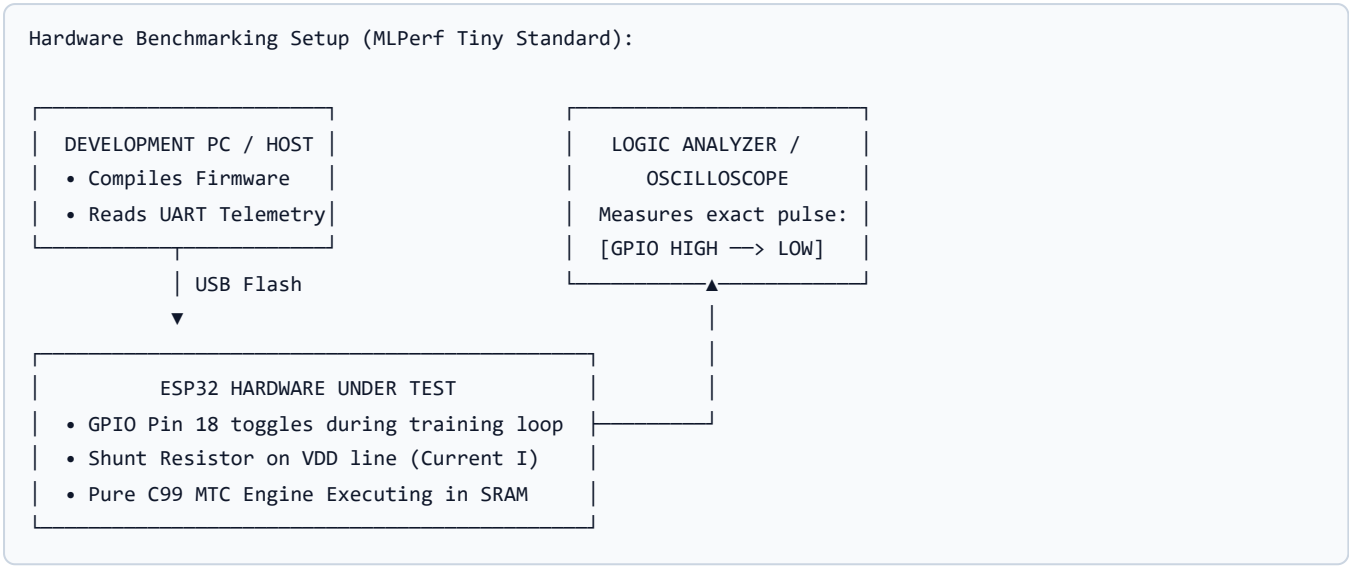
---

2. Step-by-Step Workflow

1. **Benchmark Suite Selection:** Select a reference task (e.g., Anomaly Detection using a deep autoencoder or Keyword Spotting).
2. **Target Device Flashing:** Compile the model and runtime onto the target microcontroller (e.g., ESP32 or STM32).
3. **GPIO Pin Toggling:** Set a hardware GPIO pin HIGH immediately before inference/training begins, and pull it LOW immediately when finished.
4. **Measurement Acquisition:** Use an oscilloscope or digital logic analyzer connected to the GPIO pin to measure exact execution time with microsecond accuracy.
5. **Power Profiling:** Measure current draw across a shunt resistor to calculate total energy in millijoules: $E = \int V \cdot I, dt$.
6. **Watermark Extraction:** Measure peak stack and heap usage to record the true peak SRAM watermark.

---

3. Diagram to Draw in Your Notebook



---

4. Key Formulas & Methodology

1. Energy Consumption per Cycle:

$$E = V_{\text{dd}} \times \int_{0}^{t_{\text{exec}}} I(t), dt$$

- *Explanation:* Total energy consumed during execution equals supply voltage V_{dd} multiplied by the integral of current draw $I(t)$ over execution time t_{exec}.

2. Peak SRAM Watermark Rule:

M_{\text{peak}} \le M_{\text{physical_SRAM}}

- *Explanation:* If peak memory usage exceeds physical RAM even for a microsecond, the processor halts with a memory fault.

---

5. Viva / Exam Questions for Ajay

- **Q: Why is accuracy alone not enough to evaluate an embedded neural network?**
  Answer: Because embedded systems have strict physical constraints: peak SRAM, Flash storage, execution latency, and power budget. A highly accurate model that exceeds SRAM or drains the battery cannot be deployed.
- **Q: How does MTC verify its performance against the MLPerf Tiny philosophy?**
  Answer: MTC evaluates its generated C code on real ESP32 hardware, logging exact epoch latency over high-speed UART and proving peak SRAM stays strictly within our ≤ 64 KB budget.

Paper 5: Edge Intelligence: Paving the Last Mile of AI with Edge Computing (IEEE Survey)

- **Authors:** Zhi Zhou, Xu Chen, En Li, Liekang Zeng, Ke Luo, Junshan Zhang
- **Venue:** Proceedings of the IEEE, 2019
- **Direct Link:** https://arxiv.org/abs/1905.10083
- **Direct PDF:** https://arxiv.org/pdf/1905.10083.pdf

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** The explosion of IoT devices (cameras, smart meters, industrial sensors, medical wearables) generates trillions of gigabytes of data every day. Routing all this raw data to centralized cloud data centers causes

network congestion, unacceptable latency for safety-critical systems (autonomous braking, medical alarms), and massive cloud computing bills.

- **The Breakthrough Idea:** This IEEE survey provides the definitive theoretical framework for **Edge Intelligence**, breaking it down into:
  1. **AI on Edge:** Running machine learning algorithms directly on decentralized edge nodes.
  2. **AI for Edge:** Using machine learning to optimize edge network configurations.
  3. **The Computing Spectrum:** Moving computation along a spectrum from Cloud Data Centers $\rightarrow$ Edge Gateways $\rightarrow$ On-Device Tiny Microcontrollers.

---

2. Step-by-Step Workflow

1. **Data Generation:** Physical phenomena are captured by ubiquitous edge sensors.
2. **Hierarchical Filtering:** Low-level feature extraction and simple classification happen at the lowest microcontroller tier.
3. **Decentralized Decision-Making:** Critical actions (e.g., shutting down an overheating industrial motor) are executed locally in sub-milliseconds.
4. **Selective Uplink:** Only summarized metadata, anomalies, or model weight updates are transmitted upstream, reducing network traffic by > 99%.
5. **Federated Aggregation:** Edge devices periodically synchronize their learned model updates without ever exposing raw private user data.

---

3. Diagram to Draw in Your Notebook

```
The Edge Intelligence Computing Hierarchy:

   [ CLOUD LEVEL ]         Centralized Cloud Data Centers (AWS / GCP)
          ▲                • Massive GPU Clusters • Terabytes of Storage
          | Low-Bandwidth  • Long-term model training & fleet analytics
          | Metadata Only
   [ GATEWAY LEVEL ]       Edge Gateways / Fog Nodes (Raspberry Pi / Routers)
          ▲                • Moderate Compute • Local Area Aggregation
          | Sensor Telemetry
          |
  [ ON-DEVICE LEVEL ]     TINY EMBEDDED MICROCONTROLLERS (ESP32 / MTC Tier)
                           • Ultra-low power (< 1W) • Kilobytes of SRAM
                           • Zero-latency local training & real-time actuation
```

---

4. Key Formulas & Methodology

1. End-to-End Latency Comparison:

T_{\text{cloud}} = T_{\text{trans_up}} + T_{\text{queue}} + T_{\text{compute_cloud}} + T_{\text{trans_down}}

T_{\text{edge}} = T_{\text{compute_local}}

- *Explanation:* Local edge computing eliminates network transmission latency (T_{trans}) and queue delay (T_{queue}), guaranteeing deterministic, real-time responses.

2. Bandwidth Compression Ratio:

$$R_{\text{compression}} = \frac{\text{Size}(\text{Summary Metadata / Model Weights})}{\text{Size}(\text{Raw Sensor Stream})} \ll 1$$

- *Explanation:* Local training processes raw sensor data on-chip and only communicates tiny parameter updates, reducing bandwidth demand by more than 99%.

---

5. Viva / Exam Questions for Ajay

- **Q: What is the difference between "Edge Computing" and "Cloud Computing"?**
  Answer: Cloud computing processes data in remote centralized data centers. Edge computing processes data near the data source on local hardware, providing low latency, privacy preservation, and offline capability.
- **Q: How does MTC fit into the Edge Intelligence hierarchy?**
  Answer: MTC operates at the absolute frontier of edge intelligence—the on-device microcontroller tier—enabling microcontrollers to train and adapt locally without any dependence on edge gateways or cloud servers.

Quick Reference Summary Table for Ajay

Paper No.	Short Title	Primary Problem Solved	Key Technique / Concept	Direct MTC Connection
Paper 1	**TFLM** (MLSys 2021)	Eliminating dynamic OS & `malloc` dependencies on MCUs	Static **Tensor Arena** contiguous memory buffer	MTC eliminates the runtime interpreter entirely via AOT C99 compilation
Paper 2	**TinyML Survey** (IEEE 2024)	High power, latency, and privacy risks of cloud streaming	Micro-watt on-device learning directly at sensor interface	Establishes on-device training as the ultimate goal of TinyML
Paper 3	**CMSIS-NN** (Arm 2018)	Slow software matrix multiplication on integer CPUs	**SIMD dual-MAC instructions** and Q15 fixed-point math	Inspires MTC's unrolled loops and fast integer kernel generation
Paper 4	**MLPerf Tiny** (2020)	Lack of standardized hardware-in-the-loop MCU metrics	**Multi-dimensional metrics** (SRAM watermark, latency, energy)	Sets the benchmarking methodology for MTC's ESP32 evaluation
Paper 5	**Edge Intelligence** (IEEE 2019)	Cloud bandwidth congestion & network bottlenecks	**Hierarchical compute spectrum** from Cloud to MCU node	Places MTC at the lowest, most energy-efficient layer of edge AI

Notes prepared for Ajay Sisodia (`2405111530012`) • Department of Computer Science & Engineering • G. L. Bajaj Group of Institutions, Mathura